



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A table-driven agent picks its action by looking up everything it has perceived so far in a huge table. The number of possible percept sequences grows exponentially with time, so the table gets far too big. For chess it would need about 10^{150} entries, and there are only about 10^{80} atoms in the universe. As the agent's lifetime increases, the percept history keeps growing and so does the table. We can't store it or fill it in, so we write agent programs instead. This problem is called combinatorial explosion.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept['food_here']: return 'suck'  
if percept['wall_ahead']: return 'turn_left'  
return 'forward'
```

Each one is a condition-action rule. If there is food here, the agent sucks. If there is a wall ahead, it turns left. Otherwise it moves forward. The agent only looks at the current percept and has no `__init__`, so it stores nothing.



3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The agent only sees two things, `wall_ahead` and `food_here`, so most cells look exactly the same to it. It can't tell where it is or whether it has been there before. In my run it kept walking forward, turning left at each wall, and went around the border of the grid over and over without ever reaching the middle. Because it has no percept history, it can't notice that it is repeating itself. Partial observability hides the information it needs, and no history means it can't work that information out. So it gets stuck in an infinite loop.



4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The `ModelBasedAgent` updates its memory in `_update_state` before it picks a rule.
Transition model: it updates its own position and direction based on its last action (`turn_left`, `turn_right` or `forward`), and adds the cell to `self.visited`.
Sensor model: if the percept says `wall_ahead`, it saves that wall in `self.walls`.
After that, the rules use the memory. The agent goes to cells it hasn't visited yet, and if everything nearby is visited it heads to the nearest unexplored cell. This works well. It got out of the loop and ate all the food in my tests. The downside is that it needs extra memory, and it takes more steps because it explores every cell



Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING